Experiment 3

Aim: Write a program to simulate Clock Synchronization

Theory:

Clock Synchronization in Distributed Systems

Clock synchronization in distributed systems refers to the process of ensuring that all clocks across various nodes or computers in the system are set to the same time or at least have their times closely aligned.

- In a distributed system, where multiple computers communicate and collaborate over a network, each computer typically has its own local clock.
- However, due to factors such as hardware differences, network delays, and clock drift (inaccuracies in timekeeping), these local clocks can drift apart over time.

Importance of Clock Synchronization

1. Consistency and Coherence: Clock synchronization ensures that timestamps and time-based decisions made across different nodes in the distributed system are consistent and coherent. This is crucial for maintaining the correctness of distributed algorithms and protocols.

2. Event Ordering: Many distributed systems rely on the notion of event ordering based on timestamps to ensure causality and maintain logical consistency. Clock synchronization helps in correctly ordering events across distributed nodes.

3. Data Integrity and Conflict Resolution: In distributed databases and file systems, synchronized clocks help in timestamping data operations accurately. This aids in conflict resolution and maintaining data integrity, especially in scenarios involving concurrent writes or updates.

4. Fault Detection and Recovery: Synchronized clocks facilitate efficient fault detection and recovery mechanisms in distributed systems. Timestamps can help identify the sequence of events leading to a fault, aiding in debugging and recovery processes.

5. Security and Authentication: Timestamps generated by synchronized clocks are crucial for security protocols, such as in cryptographic operations and digital

signatures. They provide a reliable basis for verifying the authenticity and temporal validity of transactions and messages.

Types of Clock Synchronization in Distributed Systems

1. Physical clock synchronization

In distributed systems each node operates with its clock, which can lead to time differences. However the goal of physical clock synchronization is to overcome this challenge. This involves equipping each node with a clock that is adjusted to match Universal Coordinated Time (UTC) , a recognized standard. By synchronizing their clocks in this way diverse systems, across the distributed landscape can maintain harmony.

- Addressing Time Disparities: When it comes to distributed systems each node operates with its clock, which can result in variations. The goal of physical clock synchronization is to minimize these disparities by aligning the clocks.
- Using UTC as a Common Reference Point: The key to achieving this synchronization lies in adjusting the clocks to adhere to an accepted standard known as Universal Coordinated Time (UTC). UTC offers a reference for all nodes.

2. Logical clock synchronization

In distributed systems absolute time often takes a backseat to clock synchronization. Think of clocks as storytellers that prioritize the order of events than their exact timing. These clocks enable the establishment of connections between events like weaving threads of cause and effect. By bringing order and structure into play, task coordination within distributed systems becomes akin to a choreographed dance where steps are sequenced for execution.

- Event Order Over Absolute Time: In the realm of distributed systems logical clock synchronization focuses on establishing the order of events rather than relying on absolute time. Its primary objective is to establish connections between events.
- Approach towards Understanding Behavior: Logical clocks serve as storytellers weaving together a narrative of events. This narrative enhances comprehension and facilitates coordination within the distributed system.

Code:

1. Server.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class Server {
    private static final int PORT = 5000;
    private static final List<ClientHandler> clients = new ArrayList<>();
    private static final List<Long> times = new ArrayList<>();

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Clock Server started...");

            while (clients.size() < 3) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " +
clientSocket.getInetAddress());

                ClientHandler clientHandler = new
ClientHandler(clientSocket);
                clients.add(clientHandler);
                clientHandler.start();
            }

            synchronizeClocks();

        } catch (IOException e) {
            e.printStackTrace();
        } finally {
            closeAllClients();
        }
    }

    private static void synchronizeClocks() {
        try {
            for (ClientHandler client : clients) {
```

```
                client.requestTime();
            }

            long avgTime = times.stream().mapToLong(Long::longValue).sum()
/ times.size();
            System.out.println("Calculated average time: " + avgTime);

            for (ClientHandler client : clients) {
                client.adjustTime(avgTime);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    private static void closeAllClients() {
        for (ClientHandler client : clients) {
            client.closeConnection();
        }
    }

    static class ClientHandler extends Thread {
        private final Socket socket;
        private PrintWriter output;
        private BufferedReader input;

        public ClientHandler(Socket socket) {
            this.socket = socket;
        }

        public void run() {
            try {
                input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
                output = new PrintWriter(socket.getOutputStream(), true);
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
```

```
        public void requestTime() throws IOException {
            output.println("GET_TIME");
            long clientTime = Long.parseLong(input.readLine());
            System.out.println("Received time from client: " +
clientTime);
            times.add(clientTime);
        }

        public void adjustTime(long newTime) {
            output.println("SET_TIME " + newTime);
        }

        public void closeConnection() {
            try {
                socket.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}
```

2. Client.java

```
import java.io.*;
import java.net.*;
import java.util.Random;

public class Client {
    public static void main(String[] args) {
        String SERVER_HOST = "127.0.0.1"; // Change if needed
        int SERVER_PORT = 5000;

        try (Socket socket = new Socket(SERVER_HOST, SERVER_PORT);
             BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
             PrintWriter output = new
PrintWriter(socket.getOutputStream(), true)) {
```

```
            long localTime = System.currentTimeMillis() + new
Random().nextInt(10000) - 5000; // Simulated time drift
            System.out.println("Client's initial time: " + localTime);

            while (true) {
                String serverMessage = input.readLine();
                if (serverMessage == null) break;

                if (serverMessage.equals("GET_TIME")) {
                    output.println(localTime);
                } else if (serverMessage.startsWith("SET_TIME")) {
                    long newTime = Long.parseLong(serverMessage.split("
")[1]);
                    localTime = newTime;
                    System.out.println("Client's adjusted time: " +
localTime);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Output: